Hugo beyond Markdown: content graphs, build pipelines, and theme architecture

Contents

1. Read a Hugo project **5**
1.1 Hello, world . . . 5
1.2 Project layout . . . 6
1.3 Pages and front matter . . . 7
1.4 Template actions . . . 8
1.5 Dot and dollar . . . 10
1.6 Units of reuse . . . 11
1.7 Look-alikes, and why both exist . . . 12
1.8 Reading output and commands . . . 13
1.9 Everyday vocabulary . . . 13

2. Replace the converter model **14**
2.1 The weak model and the useful model . . . 14
2.2 The two examples as different projections . . . 15
2.3 Execution, state, and repetition . . . 16
Exercise 1 — predict the boundary . . . 17
A useful question for an implementation agent . . . 17

3. Model a site as content, not files **17**
3.1 Page kinds are semantic inputs to dispatch . . . 17
3.2 Sections, types, layouts, and taxonomies are not synonyms 19
3.3 Front matter is control data, not just metadata . . . 20
3.4 Generated pages join the same graph . . . 20
Exercise 2 — choose representations . . . 22
A useful question for an implementation agent . . . 22

4. Predict template dispatch **22**
4.1 First discard pre-0.146 folklore . . . 22
4.2 Dispatch is a specificity calculation . . . 23
4.3 Base templates and blocks are a protocol . . . 24
4.4 The dot is an explicit context value . . . 25
4.5 Partials, views, and render hooks solve different dispatch problems . . . 26
Exercise 3 — predict the override . . . 27
A useful question for an implementation agent . . . 27

5. Treat templates as queries over a build graph **27**
5.1 Query pages instead of duplicating indexes . . . 27

5.2 Choose the data scope that matches ownership 28
5.3 Remote data creates a distributed build 28
5.4 Static projection versus live application 30
5.5 Mutable template state is scoped and concurrent 30
Exercise 4 — choose a failure policy 30
A useful question for an implementation agent 31

6. Build an asset pipeline, not a copy directory **31**
6.1 Four resource origins 31
6.2 CSS and JavaScript are dependency pipelines 32
6.3 Let the owner of an image own the expensive transformation 32
6.4 Caches are acceleration, not authority 33
Exercise 5 — review an image proposal 34
A useful question for an implementation agent 34

7. Design theme boundaries **34**
7.1 A theme is a component layer, not a coat of paint 34
7.2 Override one seam before forking the whole theme 35
7.3 Choose a customisation strategy by change pressure 36
7.4 Theme components compose local layers 37
7.5 Hugo Modules add versioned distribution and mounts . . . 38
7.6 Configuration merging is deliberately constrained 39
7.7 How to inspect an inherited theme 39
Exercise 6 — choose the theme boundary 40
A useful question for an implementation agent 40

8. Publish more than web pages **40**
8.1 One page can have several outputs 40
8.2 Render hooks turn Markdown constructs into site policy . 41
8.3 Multilingual output is another site dimension 42
8.4 Headless bundles separate storage from routing 42
Exercise 7 — design a partner output 43
A useful question for an implementation agent 43

9. Operate and diagnose the build **43**
9.1 Development server, production build, and deployment are distinct 43
9.2 A diagnostic sequence starts from the symptom 44
9.3 Performance evidence has Hugo-specific traps 46
9.4 Know which inputs are trusted 46

9.5 Upgrade with deprecations promoted to evidence 47
Exercise 8 — diagnose rather than thrash 48
A useful question for an implementation agent 48

10. Review and verify a change **48**
10.1 A successful build proves only a subset 48
10.2 A proportionate evidence stack 49
10.3 Test invariants, not implementation trivia 49
10.4 Review plausible generated mistakes 50
10.5 Decide what to delegate and what to retain 51
Exercise 9 — review the whole proposal 52
A useful question for an implementation agent 52

11. Worked reasoning **52**
Exercise 1 52
Exercise 2 53
Exercise 3 53
Exercise 4 53
Exercise 5 54
Exercise 6 54
Exercise 7 54
Exercise 8 55
Exercise 9 55

12. Fixture guide **56**

13. Compact field reference **57**
Choose the mechanism 57
Retained causal rules 58

Sources and version boundary **58**

Hugo 0.168.0 · Linux · researched 21 October 2026, reading primer 3 November 2026

This tutorial treats Hugo as a programmable publishing compiler, not as a Markdown converter. Two small sites provide the running examples:

- **Green Horizon**, an environmental advocacy organisation connecting campaigns, places, issues, actions, and evidence; and
- **Lens Vault**, a static public front end whose album records and image renditions originate in a photo-management backend.

The examples are deliberately modest visually. Their job is to expose the architecture that could support a far richer design. You should finish able to explain where Hugo gets its leverage, choose what belongs in content, templates, themes, data, or a live service, and review an implementation without writing every template by hand.

Allow 2–3 hours including the exercises, with chapter 1 a reading primer to skim and return to. Each exercise is a prediction, design decision, or review rather than a syntax drill. Worked reasoning is collected near the end.

1. Read a Hugo project

This chapter is a phrasebook: enough to recognise each part of a Hugo project and know what to look up. It is for reading, not for learning to write templates. Most items get a sentence and a small example, usually a fragment. Why things behave as they do comes in later chapters, so skim this now and return when a construct looks unfamiliar.

1.1 Hello, world

Three files make a site:

```
hugo.toml                 title = "Hello"
content/_index.md         the home page's text
layouts/home.html         the home page's template
```

```
---
title: Hello, world
```

```
---
Some *Markdown*.
```

```
<h1>{{ .Title }}</h1>
{{ .Content }}
```

Running `hugo` with no arguments builds into `public/`, where `public/index.html` contains:

```
<h1>Hello, world</h1>
<p>Some <em>Markdown</em>.</p>
```

A site like this also gets RSS feeds and a sitemap unless told otherwise. This one also logs `WARN found no layout file for "html" for kind "taxonomy"`, because Hugo creates the default `tags` and `categories` lists and nothing here can render them.

1.2 Project layout

Some names are fixed by Hugo, and most of them hold a different kind of input (directory structure).

Path	What it holds
`hugo.toml`	Site configuration; also `.yaml` or `.json`, or a `config/` directory
`content/`	Pages, in a tree that becomes the site's sections
`layouts/`	Templates
`assets/`	Files that templates process, such as CSS to fingerprint
`static/`	Files copied to the output as they are
`data/`	YAML, JSON or TOML that templates read
`i18n/`	Translated strings
`archetypes/`	Templates for `hugo new content`
`themes/NAME/`	A theme, laid out like a project
`public/`	The output; normally not committed

Path	What it holds
`resources/_gen/`	A cache of processed assets
`go.mod`	Present when the project or theme uses Hugo Modules
`theme.toml`, `exampleSite/`	A theme's description, and a demonstration site

Inside `layouts/`, the current names are `baseof.html`, `home.html`, `section.html`, `page.html`, `taxonomy.html`, `term.html`, `_partials/`, `_shortcodes/` and `_markup/`. **Older themes use `_default/single.html`, `_default/list.html`, `partials/` and `shortcodes/`**, the layout before 0.146, and chapter 4 explains the difference. `single.html`, `list.html` and `all.html` also work at the top of `layouts/` in the current scheme, as fallbacks for single pages, lists and everything. `404.html` and `robots.txt` render those special files. An older theme's `terms.html` and `taxonomy.html` meant what `taxonomy.html` and `term.html` mean now.

1.3 Pages and front matter

A content file opens with **front matter**, whose fence gives its format: `---` for YAML, `+++` for TOML, and `{` for JSON.

```
---
title: One
date: 2026-01-01
draft: true
weight: 2
slug: first
tags: [a, b]
params:
  urgency: high
---
Body text.
```

`draft: true` keeps a page out of a build unless `--buildDrafts` is given. `slug` replaces the last part of the URL, and `url` the whole path. `weight` orders pages. `tags` works because `tags` is a configured **taxonomy**. Your own keys belong under `params`, and templates read them

as `.Params.urgency` (front matter). Others you will meet are `publishDate` and `expiryDate`, which hide a page outside those times; `aliases`, for old URLs that redirect; `cascade`, which passes values to descendant pages; and `headless: true` on a bundle that should have no page of its own.

Three file names decide what a page is (page bundles):

Path	Meaning
`content/about.md`	A plain page
`content/posts/one/index.md`	A **leaf bundle**: one page, with sibling files as its resources
`content/posts/_index.md`	A **branch bundle**: the page for the section itself

The underscore matters. `_index.md` describes a list of pages, and `index.md` is a single page.

Page kinds. A content or list page has a `.Kind`: `home`, `section`, `page`, `taxonomy` (such as `/tags/`) or `term` (such as `/tags/a/`). This decides which template applies. Special pages have kinds of their own, such as `404`.

1.4 Template actions

Templates are Go's `html/template` language plus Hugo's functions. Everything between `{{` and `}}` is an **action**, and the rest is copied out (templates).

Form	Meaning
`{{ .Title }}`	A field of the current context
`{{ $n := 0 }}`	Declare a variable
`{{ $n = add $n 1 }}`	Assign to it; `add` is a function, called with spaces, not parentheses
`{{ if and (gt $n 0) .IsHome }} … {{ else if … }} … {{ else }} … {{ end }}`	Branch; operators are functions, as with `eq`, `ne`, `in` and `len`

Form	Meaning
`{{ range .Pages }} … {{ else }} … {{ end }}`	Loop, with `else` when empty
`{{ range $i, $p := .Pages }} … {{ end }}`	Loop, naming the index and value
`{{ with .Params.logo }} … {{ else with .Alt }} … {{ else }} … {{ end }}`	Run if the value is non-empty
`{{/* a comment */}}`	Not copied to the output
`{{- .Title -}}`	Remove the whitespace either side of the action

Pipes. `|` passes a value as the *last* argument of the next function, so `{{ "a" | upper }}` means `{{ upper "a" }}`, and a chain reads left to right:

```
{{ with resources.Get "css/main.css" }}
  {{ with . | minify | fingerprint }}
    <link rel="stylesheet" href="{{ .RelPermalink }}">
  {{ end }}
{{ end }}
```

The outer `with` is the guard. Piping a missing resource straight into `minify` fails the build rather than skipping the block.

Dots mean two things. `.Foo` is a field or method of the context, and may take arguments, as in `.Resources.Get "x"`. `foo.Bar` with no leading dot is a function in a namespace: `strings.ToLower`, `transform.Unmarshal`, `debug.Dump`. Many functions have both a short and a namespaced name, so `safeHTML` is `safe.HTML` and `where` is `collections.Where`.

Data structures. `dict "a" 1` builds a map and `slice 1 2 3` a list. `index LIST 1` reads an element, and `$d.a` a map's key. `default "text" .Params.x` supplies a fallback, and `append` adds to a list. `{{ T "key" }}` looks up a translation, and `relref` a link to a page. Common queries are `where .Pages "Section" "posts"`, `first 3 .Pages`, and `sort .Pages "Title" "desc"`.

You will also meet these often in themes:

Form	Meaning
`printf "%s-%d" "a" 1`	Format a string
`.Date.Format "2006-01-02"`	Format a date; Go writes the layout using that exact reference date
`now`, `jsonify`	The current time, and a value as JSON
`relURL`, `absURL`	Apply `baseURL` to a path
`markdownify`, `.RenderString`	Render a string as Markdown
`site.Menus.main`	Menu entries, from configuration or a page's `menu` front matter key
`.Paginate`, `.Paginator`	Split a list into numbered pages
`site.GetPage "/posts"`	Find a page by path
`.Pages.ByDate`	One of several ready-made sorts
`.Language`, `.Translations`	The page's language, and its other versions
`hugo.IsProduction`, `hugo.Environment`	Which environment this build is for

Escaping. Output is escaped according to where it lands, so `{{ "<b>" }}` prints `<b>`. `safeHTML` and its relatives switch this off, so each use deserves a question. `.Content` and `.Summary` are already marked as safe HTML.

Errors. `errorf "text %s" $x` logs an error, rendering carries on, and the build fails when it finishes, so `public/` may hold output from a failed build. `warnf` only logs. A field that does not exist on a page, such as `.Nope`, fails that page with `can't evaluate field Nope`, while a missing key in a map, such as `.Params.nope`, is silently empty. `try` catches a failure in an expression and exposes `.Err` and `.Value`.

1.5 Dot and dollar

`.` is the current context, and it changes. At the top of a template it is usually the page. Inside `range` and `with` it becomes the current element. Chapter 4 gives this the attention it needs.

`$` is the context the template started with, so inside a loop `.Title` is the element's title and `$.Title` the page's.

`.Site` **and** `site`**.** `site` is a function available anywhere, and `.Site` a field that needs a page as context. `.Page`, available on most objects, reaches the page from a shortcode or render hook.

1.6 Units of reuse

Form	Meaning
`{{ partial "card.html" . }}`	Run `_partials/card.html`, with `.` as its context
`{{ partial "card.html" (dict "page" . "n" 1) }}`	Pass several values, as a map
`{{ return (mul . 2) }}`	Inside a partial: return a value, like a function
`{{ partialCached "nav.html" . }}`	The same, reusing one result
`{{ block "main" . }} … {{ end }}`	In `baseof.html`: a slot, with default content
`{{ define "main" }} … {{ end }}`	In a page template: fill that slot
`{{ template "main" . }}`	Call a named `define`; often the slot in `baseof.html`, with no default

Base templates. A `home.html` or `page.html` usually contains only `define` blocks, and `baseof.html` holds the surrounding page. Text outside `define` in such a file is a known trap (chapter 4).

Shortcodes are the reuse form for page authors, and live in content rather than templates (shortcodes):

```
{{< note kind="x" >}}inner text{{< /note >}}
{{% note kind="y" %}}inner *text*{{% /note %}}
```

`_shortcodes/note.html` reads `.Get "kind"` and `.Inner`.

Render hooks are templates in `_markup/`, such as `render-image.html`, that replace how a Markdown construct renders throughout the site (chapter 8).

1.7 Look-alikes, and why both exist

`{{< >}}` **and** `{{% %}}`**.** Output from the first form goes into the page as it is. Output from the second is treated as Markdown, so raw HTML in it is dropped by default. In a test, the `%` form's `<aside>` came out as `<!-- raw HTML omitted -->`.

`_index.md` **and** `index.md`**.** A list page and a single page, as above.

`assets/` **and** `static/`**.** Templates can transform files in `assets/`, and only what they use is published. `static/` is copied whole and is out of their reach. A page bundle's own files are a third kind, **page resources**, reached through `.Resources`.

`.Permalink` **and** `.RelPermalink`**.** The first includes the scheme and host from `baseURL`, and the second only the path.

`.Pages` **and** `.RegularPages`**.** `.Pages` is a list page's children, including subsections. `.RegularPages` has only its pages of kind `page`, without descending into subsections. `site.RegularPages` has every such page in the site.

`.Content`**,** `.Summary` **and** `.Plain`**.** The rendered body, a teaser, and the body without markup.

`.Params` **and** `site.Params`**.** A page's own values, and the `[params]` table in `hugo.toml`.

`.Type`**,** `.Section` **and** `.Kind`**.** Chapter 3 separates these. `.Kind` is one of the five kinds above, `.Section` comes from the top-level content directory, and `.Type` can be set to choose templates and defaults to the section.

`.Store` **and** `.Scratch`**.** Both hold mutable per-page values, and `.Store` is the current choice.

Theme and module. A theme is a directory listed in `theme =`. A **Hugo Module** is a versioned component fetched with Go's tooling, and appears under `[module]` with `[[module.imports]]`. `[[…]]` is TOML's notation for one item of a list. Chapter 7 compares them.

`hugo server` **and** `hugo`**.** The first builds in memory, serves with live reload and defaults to the `development` environment. The second writes to `public/` for `production`. Chapter 9 gives the consequences.

1.8 Reading output and commands

```
                  | EN
------------------+----
 Pages            | 13
 Non-page files   |  1
 Static files     |  1
 Processed images |  0
```

Build totals count more than your `content/` files, because `Pages` includes the sections, taxonomies and terms Hugo generates.

A template failure names the file, line and column, as in `layouts/term.html:1:29`, and `WARN` lines do not stop the build unless `--panicOnWarning` is given. Fingerprinted output files carry a hash in the name, as in `css/main.min.15c42ab7….css`.

Command	Meaning
`hugo`, `hugo build`	Build into `public/`; the two are the same
`hugo server -D`	Serve locally, with drafts
`hugo new content posts/x.md`	Create a page from an archetype
`hugo list all`	List pages and their dates as CSV
`hugo config`, `hugo config mounts`	Show effective settings and component layers
`hugo mod get`, `hugo mod graph`	Fetch and inspect modules
`hugo --gc --minify`	A common production build

1.9 Everyday vocabulary

Term	Meaning
Section	A top-level `content/` directory, or a nested one with `_index.md`
Taxonomy, term	A classification such as `tags`, and one value such as `a`
Bundle, resource	A page's directory, and the files in it

Term	Meaning
Headless	A bundle with no page of its own, used only as data
Content adapter	`_content.gotmpl`: a template that creates pages from data
Output format	One page rendered in another form, such as JSON or RSS
Mount	A rule mapping a directory into Hugo's virtual file system
Fingerprint	A content hash in a file's name, for safe long-term caching
Archetype	A template for a new content file
Goldmark	Hugo's Markdown renderer, configured under `[markup]`

2. Replace the converter model

2.1 The weak model and the useful model

The weak model is:

```
Markdown file ──> HTML file
```

That model predicts little beyond a simple article. Hugo's useful model is:

```
project files       theme/module layers       remote inputs
     │                      │                       │
     └──────────────────> virtual component file system
                                    │
content + generated pages ──> typed page graph
                                    │
                    template dispatch + queries
                                    │
                resource transformation pipelines
                                    │
                 HTML, JSON, RSS, CSS, images, …
```

There are five interacting sources of leverage:

1. **A typed page graph.** Home, regular pages, sections, taxonomies, and terms are different page kinds with relationships and collections. A page is not merely a document body.
2. **A layered component file system.** The project, themes, and Hugo Modules contribute content, templates, assets, data, translations, static files, and archetypes. A higher layer can replace one file from a lower layer.
3. **Specificity-based template dispatch.** Hugo chooses a template from the page's kind, logical path, type, requested output, language, and optional layout. This behaves more like multi-method dispatch than a single renderer.
4. **Build-time querying and transformation.** Templates query page collections and structured data. Resource pipelines resize images, build CSS or JavaScript, fingerprint results, and cache transformations.
5. **Multiple projections.** One logical page may produce HTML, JSON, RSS, or a specialised language variant without duplicating the source record.

Hugo calls the seven directories that modules can contribute—`archetypes`, `assets`, `content`, `data`, `i18n`, `layouts`, and `static`—*components*. Modules and themes are therefore not plug-ins running arbitrary callbacks. They contribute material to the same build model as the project (module introduction, directory structure).

This explains much of Hugo's speed and its boundary. The build has all the inputs it needs, computes a finite result, and exits. There is no Hugo process handling a visitor's request in production. User authentication, an album edit, live stock, personalised search, and database transactions need a browser-side application or a service. Hugo can publish a snapshot or a client that talks to that service; it does not become the service.

2.2 The two examples as different projections

Green Horizon can derive several views from a campaign record:

- a campaign detail page;
- cards on the home page and campaign index;
- lists grouped by issue or place;
- an evidence register stored with that campaign;

- perhaps a JSON action feed for partner organisations; and
- translated variants sharing the same identity.

Lens Vault can turn one album record into:

- an album route and gallery card;
- responsive image markup from backend-provided renditions;
- a machine-readable catalogue for client-side search; and
- feeds or collection manifests.

Neither domain is naturally a reverse-chronological blog. The page graph and template dispatcher do not require it to be one.

2.3 Execution, state, and repetition

A production build reads configuration and component layers, constructs sites and their pages, resolves templates, executes transformations, and writes a destination directory. Template rendering is parallel. Resource and partial caches can avoid repeated computation, but the published directory remains the material result.

The build is not a database transaction. A failed build may have written some files to its destination. Nor does a normal build clear old files first: it overwrites matching paths and otherwise leaves stale files in place. Hugo documents this as deliberate; use the current `build .cleanDestinationDir` configuration or `--cleanDestinationDir` when the destination is disposable (basic usage). A safer deployment builds into a new directory and promotes it atomically only after validation.

On a second identical build:

- logical output should be equivalent;
- cached transformations may make execution faster;
- generated timestamps or code using `now` may differ; and
- stale files remain unless the destination is cleaned.

Reproducibility is therefore a design property, not an automatic consequence of using a static generator. Pin Hugo and dependency versions, control time where needed, make remote inputs explicit, and compare outputs that matter.

Exercise 1 — predict the boundary

The photo backend has 400,000 private images. A proposal says: "Hugo will fetch the API during deployment, generate the public albums, handle a signed-in owner's edits, and update the gallery immediately after each edit." Divide that sentence into work Hugo can own and work another component must own. What is the freshness contract of the Hugo-owned part?

Worked answer

A useful question for an implementation agent

> Draw the inputs and derived outputs for one campaign or album, and identify which values exist only at build time, which are published, and which require runtime state.

A strong answer names actual source files or API fields, derived routes and assets, the rebuild trigger, cache and failure behaviour, and the service that owns runtime operations. "Hugo can call APIs and is very fast" is not evidence of a sound boundary.

3. Model a site as content, not files

3.1 Page kinds are semantic inputs to dispatch

Hugo has five page kinds:

Kind	Meaning	Typical Green Horizon example
`home`	Root of a site/language	Organisation landing page
`page`	Regular content page	One campaign
`section`	Branch of the content tree	Campaign collection
`taxonomy`	Vocabulary index	All issues
`term`	One vocabulary value	Campaigns concerning water

These are first-class page objects, not merely routes (page kinds). Templates can query their children, ancestors, terms, resources, alternate outputs, and translations.

Two filenames carry disproportionate meaning:

- `index.md` creates a **leaf bundle**, a regular page that may own resources but cannot have descendant pages.
- `_index.md` creates or configures a **branch bundle**, such as home or a section, which may own resources and have descendants.

For example:

```
content/
├── _index.md                          # home: branch
└── campaigns/
    ├── _index.md                      # section: branch
    └── living-estuary/
        ├── index.md                   # campaign: leaf
        └── evidence.json              # resource owned by that
↪   campaign
```

A top-level content directory is a section. A nested directory becomes a section when it has `_index.md`; directory nesting alone is not enough (sections, page bundles). This is a common review failure because the resulting URL may still look plausible.

The Green Horizon fixture keeps `evidence.json` beside `index.md`. It is a page resource and is visible through that campaign's `.Resources`, not as global site data. That ownership is valuable: moving or deleting the campaign moves or deletes its evidence with it. Its page template turns the resource into typed template data:

```
{{ with .Resources.Get "evidence.json" }}
  {{ $evidence := . | transform.Unmarshal }}
  {{ if not $evidence }}
    {{ errorf "Campaign %q has an empty evidence register"
↪   $.File.Path }}
  {{ end }}
  {{ range $index, $record := $evidence }}
    {{ if not (and $record.claim $record.source
↪   $record.reviewed) }}
      {{ errorf "Campaign %q evidence record %d is incomplete"
        $.File.Path (add $index 1) }}
    {{ end }}
    <li><strong>{{ $record.claim }}</strong> - {{ $record.source
↪   }}</li>
  {{ end }}
{{ else }}
```

```
  {{ errorf "Campaign %q has no evidence.json" $.File.Path }}
{{ end }}
```

The explicit checks make "every campaign has at least one complete, traceable evidence record" a build invariant. They are not a full JSON Schema: a real site may also validate date format and allowed fields before Hugo runs. If evidence is optional, an empty state would be more honest. The important choice is that absence and malformed required records have specified semantics.

3.2 Sections, types, layouts, and taxonomies are not synonyms

Keep these axes separate:

- **Section** comes from the content tree.
- **Kind** describes the structural role (`page`, `section`, and so on).
- **Type** selects a content/template family. By default it is derived from the top-level section, but front matter can override it.
- **Layout** is an explicit request for a custom template.
- **Taxonomy** represents a many-to-many classification relation.

Do not put a business field named `type` at the top level of front matter if it does not intentionally affect template lookup. Put it under `params`. An agent that writes `type: urgent` to classify campaigns may silently redirect every such page into an `urgent` template family.

Green Horizon configures two taxonomies:

```
[taxonomies]
  issue = 'issues'
  place = 'places'
```

A campaign declares:

```
issues: [water, biodiversity]
places: [north-estuary]
```

Hugo derives a taxonomy page for all issues and a term page for each value. The campaign remains the source of truth; the reverse indexes are projections. Taxonomies are flat classifications, not a general graph

database. Model a relationship as a taxonomy when term-centred navigation and many-to-many membership are useful. Use explicit page references or identifiers when the edge has attributes, ordering, validity dates, or referential-integrity needs (taxonomies).

3.3 Front matter is control data, not just metadata

Front matter can affect publication dates, aliases, URL construction, outputs, template selection, taxonomies, and descendant defaults. A branch page's `cascade` can supply values to descendants and can target them by logical path, kind, environment, or site (front matter, cascade).

For example, a campaign section might default all descendants to a campaign type and a review owner. A cascade is useful when the default belongs to the branch's policy. It becomes dangerous when inherited values are hard to see or when a child that should override them does not. Ask an implementation to show the effective value on representative descendants, not merely the cascade source.

Separate fields by responsibility:

Field	Owner	Reason
Campaign title and public summary	Content team	Editorial fact
Issue/place membership	Information architecture	Navigation relation
`layout` override	Theme maintainer	Presentation dispatch
Backend album ID	Photo system	Stable external identity
Signed rendition URL	Runtime backend	Secret/expiring capability; do not publish at build time
Public rendition dimensions and URL	Backend snapshot	Stable input to static responsive markup

3.4 Generated pages join the same graph

Lens Vault does not maintain one Markdown stub per backend album. Its checked-in API snapshot lives at `assets/api/albums.json`; the

content adapter `content/albums/_content.gotmpl` maps each record into a Hugo page:

```
{{ $catalog := slice }}
{{ with resources.Get "api/albums.json" }}
  {{ $catalog = . | transform.Unmarshal }}
{{ else }}
  {{ errorf "Missing assets/api/albums.json" }}
{{ end }}

{{ range $catalog }}
  {{ $page := dict
    "kind" "page"
    "path" .id
    "title" .title
    "summary" .summary
    "params" (dict "album" .)
  }}
  {{ $.AddPage $page }}
{{ end }}
```

Content adapters can add pages and resources from local or remote structured data. The resulting pages participate in ordinary lookup, collections, and outputs. An adapter is thus a front-end import boundary, not a second templating system (content adapters).

Consequential constraints deserve design attention:

- the adapter runs while the page graph is being constructed, so its `Site` is not fully initialised and page-dependent methods are unavailable;
- colliding paths from an adapter and a content file have indeterminate results because processing is concurrent; use `--printPathWarnings` and validate source identity;
- remote input turns availability and freshness into build concerns; and
- content-adapter output is treated as untrusted content, while the adapter template itself is trusted build code.

The checked-in snapshot makes the fixture deterministic and reviewable. A real pipeline could fetch the API into a versioned artefact before Hugo runs, or use `resources.GetRemote` inside the adapter. The

first approach separates fetch, schema validation, approval, and rendering; the second is simpler but couples the build to the API. That is an architectural choice, not a syntax preference.

Exercise 2 — choose representations

Green Horizon adds public events with a venue, campaign, start/end time, accessibility information, and possible cancellation. Decide which should be a section, page bundle, taxonomy, ordinary parameter, or explicit page reference. Would you put events inside campaign bundles or in their own top-level section? State the query and ownership pressures that decide.

Worked answer

A useful question for an implementation agent

> For each content relationship, show its stable identity, source of truth, reverse-navigation requirement, validation rule, and generated routes.

A strong answer distinguishes structural pages from classification and explicit references, demonstrates `index.md` versus `_index.md`, and includes collision or missing-reference checks. A weak answer presents a directory tree without explaining the queries it enables.

4. Predict template dispatch

4.1 First discard pre-0.146 folklore

Hugo 0.146.0 replaced the template system. For Hugo 0.168.0, the representative layout is:

```
layouts/
├── baseof.html
├── home.html
├── page.html
├── section.html
├── taxonomy.html
├── term.html
├── albums/
```

```
│   ├── page.html
│   └── section.html
├── _partials/
├── _shortcodes/
└── _markup/
```

Older tutorials commonly use `layouts/_default/single.html`, `layouts/_default/list.html`, `layouts/partials`, and `layouts/index.html` for home. Hugo retains some compatibility, but these are not the current model. The official overview records the migration and current identifiers (new template system). This matters particularly when asking an agent trained on mixed-age examples.

4.2 Dispatch is a specificity calculation

For a requested page/output pair, lookup considers:

- a custom `layout` from front matter;
- page kind;
- standard layout (`single`, `list`, or catch-all `all`);
- output format;
- language;
- media type/suffix;
- logical page path; and
- content type.

Page-path matches can be nested. Thus `layouts/albums/page.html` specialises regular pages below `/albums`, while `layouts/page.html` remains a general fallback. A `type` override effectively substitutes the top-level section in lookup. The project and all theme/module layers participate; lookup seeks the most specific candidate across them, with the project winning an otherwise equal match (template types, lookup overview).

Some predictions for the fixtures:

Request	Selected main template	Why
Green Horizon `/` HTML	theme `layouts/home.html`	Home kind; project supplies no candidate

Request	Selected main template	Why
Green Horizon `/campaigns/` HTML	theme `layouts/section.html`	Section fallback
Green Horizon `/campaigns/living-estuary/`	theme `layouts/page.html`	Regular-page fallback
Lens Vault `/albums/`	project `layouts/albums/section.html`	Path-specific section
Lens Vault `/albums/marsh-at-dawn/`	project `layouts/albums/page.html`	Path-specific regular page
Lens Vault `/catalog.json`	project `layouts/home.catalog.json`	Home plus custom output format

Do not try to memorise the entire lookup table. Retain the axes, then use a tiny fixture and Hugo's inspection tools for a disputed case. `templates.Current` can expose the executing template during diagnosis.

4.3 Base templates and blocks are a protocol

A selected template is composed with a base template only when it contains at least one `define` action and otherwise contains only definitions, whitespace, and template comments. This page template participates:

```
{{ define "main" }}
  <h1>{{ .Title }}</h1>
  {{ .Content }}
{{ end }}
```

This superficially similar one does not, because it emits text outside the definition:

```
<!-- debug -->
{{ define "main" }}
  <h1>{{ .Title }}</h1>
{{ end }}
```

That HTML comment is not a Go-template comment, so the file contains something other than definitions. The template is executed on its own rather than filling the base's block. `html/template` strips the comment from the output and nothing invokes `main`, so in the advocacy fixture the page is a blank two-byte file. This is a convincing failure because the template parses, the build succeeds, and the comment looks harmless; the symptom is an empty page. The exact base-template conditions are documented under template types.

4.4 The dot is an explicit context value

Go templates are lexically terse but context-sensitive. `.` is the current value, and `range` or `with` changes it:

```
{{ range .Pages }}
  {{ .Title }}                {{/* dot is the child Page */}}
{{ end }}
```

Capture outer context before changing it, or use `$` for the root context:

```
{{ $section := . }}
{{ range .Pages }}
  <a data-section="{{ $section.Title }}" href="{{ .RelPermalink
  ↪ }}">
    {{ .Title }}
  </a>
{{ end }}
```

A partial receives only the context explicitly passed to it. Passing a small dictionary makes an interface visible:

```
{{ partial "remote-picture.html" (dict
  "cover" $album.cover
  "sizes" "(min-width: 76rem) 70rem, 92vw"
  "title" .Title
) }}
```

Inside that partial, `.cover` exists; page methods such as `.RelPermalink` do not. This is desirable when it narrows coupling. Passing the whole page is appropriate when the component is genuinely page-aware.

4.5 Partials, views, and render hooks solve different dispatch problems

Use these mechanisms deliberately:

Mechanism	Selection	Good use
Partial	Explicit name; explicit context	Header, image component, returned helper value
View (`Page.Render`)	Automatic page-aware lookup	A content card whose rendering varies by page type/path/output
Render hook	Markdown construct and page context	Site-wide links, images, headings, code blocks
Shortcode	Explicit content author invocation	Domain component intentionally embedded in content

Partials live under `layouts/_partials`. They do **not** automatically specialise for the current page kind, type, path, language, or output; a caller selects them by name. Views do use page-aware lookup. This distinction prevents a theme from accumulating conditionals in a universal `card.html` when Hugo could dispatch to type-specific views (template types).

Render hooks can turn an ordinary Markdown image into a responsive image policy, or validate and decorate links. They centralise semantics authors should not have to remember. A shortcode exposes theme implementation in content and should be reserved for an intentional content-domain vocabulary. If content outlives themes, a large shortcode dialect becomes migration debt.

`partialCached` can reuse a partial result. Every input that can change the result must be included in its variant keys. Caching a navigation partial only by language when it also highlights the current page produces correct HTML on whichever page populates the cache first and wrong HTML elsewhere. Template rendering is concurrent, and initial

cached calls may execute more than once; do not use cached partial execution as a once-only side effect (cached partials).

Exercise 3 — predict the override

The Green Horizon theme supplies `themes/field-notes/layouts/_partials/campaign-card.html`. The project supplies `layouts/_partials/campaign-card.html` with the same relative path. Predict which one the theme's `home.html` call renders. Now suppose the project instead adds `layouts/campaigns/page.html`: which routes change, and does the theme's base still apply?

Worked answer

A useful question for an implementation agent

> For these representative routes and output formats, list the winning template and base, show the higher-priority candidates, and demonstrate the result in generated output.

A strong answer uses current layout paths and evidence from a minimal build. A weak answer cites "Hugo convention" or changes several files until the page looks right without explaining dispatch.

5. Treat templates as queries over a build graph

5.1 Query pages instead of duplicating indexes

On Green Horizon's home page:

```
{{ range where site.RegularPages "Section" "campaigns" }}
  {{ partial "campaign-card.html" . }}
{{ end }}
```

The home page does not maintain a separate list of campaigns. It queries the graph. Real designs can combine `where`, `ByParam`, `GroupBy`, `Related`, term membership, dates, weights, and pagination. This is database-like in expression but not in execution: Hugo operates over

the in-memory build graph and writes a snapshot. There is no runtime query plan or transactional update.

Treat ordering as a product decision. A query that happens to inherit Hugo's default page order may appear stable until dates or weights change. Specify whether campaigns sort by editorial weight, urgency, start date, title, or a separate curated feature list. Then verify the produced order.

5.2 Choose the data scope that matches ownership

Hugo exposes several structured-data paths:

Source	Scope and lifecycle	Suitable example
Front matter	One page; part of its identity	Campaign status
Page resource	One bundle	Campaign evidence register
`data/` via `hugo.Data`	Whole build; loaded as a tree	Small shared jurisdiction codes
Global resource in `assets/`	Fetched and parsed on demand	Album API snapshot
Remote resource	HTTP input with cache/security policy	CMS or photo API response
Content adapter	Creates pages/resources during graph construction	One page per album record

Current code should use `hugo.Data`; `.Site.Data` was deprecated in 0.156.0 (data function). The entire `data/` tree is loaded for the build, whereas resources can be scoped and transformed where used. A huge catalogue placed under `data/` because "JSON belongs there" may consume memory needlessly and loses the resource pipeline's cache model.

5.3 Remote data creates a distributed build

The current error-aware pattern is:

```
{{ $data := slice }}
{{ $url := "https://photos.example/api/public-albums" }}
{{ with try (resources.GetRemote $url) }}
  {{ with .Err }}
    {{ errorf "Album fetch failed: %s" . }}
  {{ else with .Value }}
    {{ $data = . | transform.Unmarshal }}
  {{ else }}
    {{ errorf "Album API returned no resource: %s" $url }}
  {{ end }}
{{ end }}
```

`resources.GetRemote` returns `nil` for a 404; request failures otherwise fail unless handled through `try`. The returned resource is cached on disk. HTTP caching and watch-mode polling are separately configurable; by default Hugo bypasses the HTTP cache and polling is disabled, so a running development server will not detect remote changes until restarted (remote resources, HTTP cache).

A production design must answer:

1. **Freshness:** Is the site a snapshot as of its build, and what triggers the next build?
2. **Failure:** Does API unavailability fail publication, use a last-known-good snapshot, or publish an explicit degraded state?
3. **Schema:** Where is the response validated before templates dereference it?
4. **Identity:** Can an API record change its output path, and what redirects or collision checks follow?
5. **Credentials:** Can the build read only public/publishable fields, and can error output or caches expose secrets?
6. **Reproducibility:** Can the exact input behind a deployment be recovered?

The photo fixture chooses a checked-in snapshot. A robust larger system might run `fetch → validate schema → store immutable snapshot → Hugo build → validate output → promote`. That makes API recovery independent from template debugging. Direct fetching inside Hugo may be reasonable for a low-risk public API when the freshness and failure policies are explicit.

5.4 Static projection versus live application

Consider three photo behaviours:

- Public album titles and stable renditions change a few times per day. A webhook-triggered static rebuild is a good fit.
- A signed-in user edits captions and expects immediate read-after-write consistency. The browser and backend own this path.
- Public search spans only the current published catalogue. Hugo can emit a compact JSON index which browser code searches locally; a large or permission- sensitive catalogue requires a search service.

"Jamstack" does not erase these distinctions. It moves an operation to build time only when snapshot semantics are acceptable.

5.5 Mutable template state is scoped and concurrent

Prefer values flowing through pipelines and partial interfaces. When accumulation is necessary, Hugo offers local scratch state, page `Store`, site `Store`, and a global store. Broader state increases the chance of order dependence because pages render concurrently. Values populated during content rendering may also be indeterminate until `.Content` (or another content-rendering method) has been evaluated (stores, local scratch).

Do not implement cross-page validation by appending to a global store and assuming a final template sees a deterministic order. Prefer a query over the completed page graph, an adapter-level check, or a separate validation step.

Exercise 4 — choose a failure policy

The public photo API is occasionally unavailable for five minutes. Albums must never disappear merely because deployment coincides with an outage, but a deleted album must be removed within 24 hours. Design the input and cache policy, including what artefact proves which snapshot was deployed and when the build must fail.

Worked answer

A useful question for an implementation agent

> Show the normal, 404, timeout, malformed-response, and stale-cache paths for this remote input. For each, state whether publication stops and what evidence an operator sees.

A strong answer uses current `try` semantics, separates HTTP and file-cache policy, and tests the decision table. A weak answer merely wraps the call in `with`, silently turning all missing data into an empty page.

6. Build an asset pipeline, not a copy directory

6.1 Four resource origins

Do not collapse these into "files Hugo serves":

Origin	How obtained	Publication behaviour
Page resource	`.Resources.Get` on a bundle page	Scoped to page; processable
Global asset	`resources.Get` from `assets/` or a mount	Processable; published when its permalink or `Publish` is invoked
Remote resource	`resources .GetRemote`	Cached, processable, governed by HTTP security policy
Static file	Under `static/`	Copied through without the resource pipeline

The `static/` directory is for artefacts already fit to publish. The `assets/` directory is for build inputs. Referring to an asset's `.RelPermalink`, `.Permalink`, or `.Publish` causes publication; `.Content` can inline it. Hugo caches a transformation chain by its inputs and operations (Hugo Pipes).

The advocacy evidence files are page resources. They are also copied into the example output because leaf-bundle resources publish by default. If evidence is intended only as input to rendered HTML, configure the bundle's resource build behaviour or place sensitive source data elsewhere. Never assume that "the template only reads it" means "Hugo will not publish it". Inspect the output.

6.2 CSS and JavaScript are dependency pipelines

Both fixtures obtain CSS as a global resource, minify it, fingerprint it, and emit Subresource Integrity metadata:

```
{{ with resources.Get "css/main.css" }}
  {{ with . | minify | fingerprint }}
    <link rel="stylesheet" href="{{ .RelPermalink }}"
      integrity="{{ .Data.Integrity }}" crossorigin="anonymous">
  {{ end }}
{{ end }}
```

The generated filename contains the content hash. A change produces a new URL, which makes long-lived caching safe when deployment removes stale artefacts. The same resource model can bundle and tree-shake JavaScript through embedded esbuild, process PostCSS, or call Dart Sass. External tools remain dependencies of the build image and must be pinned and tested.

For Hugo 0.168.0, the old advice to install Extended for image/WebP support or modern Sass is obsolete. WebP works in every edition. Embedded LibSass is deprecated; Dart Sass is compatible with every edition. Use the standard edition unless direct cloud deployment or the deprecated embedded LibSass path is specifically needed (Linux installation and editions, module version settings).

6.3 Let the owner of an image own the expensive transformation

For local editorial images, a page bundle plus Hugo processing is powerful:

```
{{ with .Resources.Get "hero.jpg" }}
  {{ $small := .Resize "640x webp q70" }}
```

```
  {{ $large := .Resize "1280x webp q75" }}
  <img
    src="{{ $small.RelPermalink }}"
    srcset="{{ $small.RelPermalink }} 640w, {{
↪   $large.RelPermalink }} 1280w"
    width="{{ .Width }}" height="{{ .Height }}"
    alt="{{ .Params.alt }}">
{{ end }}
```

Hugo can resize, crop, convert, filter, inspect metadata, and cache results (image processing). The original dimensions still affect build memory and time, and metadata policy matters: a photo can expose location or equipment details unless intentionally removed or selected.

Lens Vault makes a different choice. The photo backend already owns originals, colour policy, access control, focal points, and rendition generation. Its snapshot provides stable public URLs and dimensions; Hugo assembles `<picture>` markup. Downloading every original into Hugo just to recreate backend renditions would duplicate authority, expand the trust boundary, and make builds fragile. A theme should accept a rendition contract, not assume all images are local page resources.

Whichever component transforms images should provide:

- intrinsic dimensions to prevent layout shift;
- meaningful alternative text policy;
- responsive candidates appropriate to actual layout widths;
- explicit cropping/focal-point behaviour;
- format fallbacks for the target browsers;
- cache and invalidation behaviour; and
- a metadata/privacy policy.

6.4 Caches are acceleration, not authority

Hugo has file caches for assets, images, remote resources, modules, and other work. `hugo build --gc` removes expired or unused cache entries; it does not clean stale files from the publish directory (file caches). A clean-cache build is a useful diagnostic when an incremental result is suspect, but repeatedly deleting caches without isolating the key or dependency hides the cause.

Exercise 5 — review an image proposal

An agent proposes that Lens Vault should fetch every `original_url` with `resources.GetRemote`, resize it at 320, 640, 1280, and 2560 pixels, and publish the results. The backend already exposes immutable public renditions at those sizes and keeps originals private. Review the proposal for ownership, security, build cost, failure behaviour, and cache invalidation.

Worked answer

A useful question for an implementation agent

> Trace one source image or stylesheet through every transformation to its published URL. Show what invalidates each cached result and what the final output contains.

A strong answer includes observed paths, dimensions or hashes, dependency/tool versions, and clean-build evidence. A weak answer calls everything "static" and does not inspect whether source data or originals were published accidentally.

7. Design theme boundaries

7.1 A theme is a component layer, not a coat of paint

A Hugo theme can contribute:

- layout selection and component templates;
- CSS, JavaScript, fonts, and image-processing policy;
- render hooks and shortcodes that extend the content language;
- default parameters, menus, media types, and output formats;
- translations and data;
- archetypes for creating content; and even
- content or static files.

It can therefore encode information architecture, accessibility behaviour, structured metadata, analytics, asset toolchains, and dependencies—not merely colours and typography. Installing a theme is closer to adopting a framework and schema than selecting a skin.

That power is why theme evaluation should start from contracts:

Contract	Questions to resolve
Content	Which sections, types, taxonomies, parameters, and resources does it assume?
Rendering	Which templates, partial interfaces, render hooks, and output formats does it own?
Configuration	Which parameters are required, defaulted, or silently ignored?
Front end	Which DOM, CSS classes, JavaScript, external tools, and browser support are assumed?
Operations	Which Hugo range, module versions, remote inputs, secrets, and build tools are required?
Evolution	Which extension points are stable, and what does an upstream upgrade overwrite or invalidate?

A screenshot proves almost none of this.

7.2 Override one seam before forking the whole theme

Hugo combines the project and themes/components by precedence. The project is first; configured theme components follow from left to right. For layouts, static files, and archetypes, an equal file path is resolved at file level. Data and translations merge more deeply (theme components).

In the advocacy fixture, the local theme owns:

```
themes/field-notes/layouts/_partials/campaign-card.html
```

and the project owns:

```
layouts/_partials/campaign-card.html
```

Calls made from the theme render the project's card. The project adds an urgency attribute and label without copying the base, home, section, taxonomy, or page templates. This is the smallest useful modification seam.

Template specificity still matters. A project's generic `layouts/page.html` does not necessarily beat a dependency's more specific `layouts/albums/page.html`; template lookup interleaves candidates across the project and themes. To replace an inherited candidate, identify the actual winning relative path and override that path. Do not infer that "project always wins" means "a less specific project template wins".

An override is a small fork of one file. It deliberately stops receiving upstream changes to that file. Record why it exists and compare it when upgrading the theme. Avoid copying a template merely to change text or colour when the theme exposes a suitable parameter or CSS custom property.

7.3 Choose a customisation strategy by change pressure

Use the least coupled strategy that expresses the design honestly:

Pressure	Usually prefer	Trade-off
Supported brand values or feature toggles	Theme parameters and project assets	Low maintenance; limited to designed extension points
One component needs different markup	Exact project override	Clear and bounded; upstream changes to that file no longer flow through
Reusable organisation header, render hooks, or search output	A narrow theme component/module before the base theme	Composable; requires namespace and compatibility discipline

Pressure	Usually prefer	Trade-off
Many structural assumptions differ	Project-owned templates or a bespoke theme	More ownership, less struggle against another theme's model
Upstream is almost right but refuses needed changes	Maintained fork	Full control; recurring merge and security work
Temporary upstream fix	Versioned fork or project override with removal condition	Keeps urgency bounded; must not become invisible permanent debt

Green Horizon demonstrates inherited theme plus override. Lens Vault owns its small set of templates directly. "From scratch" here does not mean hand-writing every HTML atom; it means the team owns the content contract and template boundaries and can still import narrow components.

For a distinctive site, starting with a visually opinionated all-purpose theme can cost more than starting with a small semantic base. Conversely, a mature theme that already supplies accessible navigation, internationalisation, SEO, and a compatible design system may remove years of incidental work. Count assumption fit and maintenance evidence, not template files.

7.4 Theme components compose local layers

A project can use multiple local themes:

```
theme = ['organisation-overrides', 'shared-components',
 ↪  'base-theme']
```

Precedence runs left to right after the project. A component can itself include other components. This is useful for capabilities such as:

- an organisation-wide header and tokens;
- a gallery and image render hook;
- a privacy-preserving analytics integration;

- search-index output; or
- common accessibility helpers.

Arbitrary complete themes rarely compose cleanly. Two themes may both expect to own `baseof.html`, the same parameters, CSS reset, or navigation model. Theme components work best when designed around narrow, documented contribution points.

7.5 Hugo Modules add versioned distribution and mounts

Hugo Modules use Go Modules for recursively versioned Git dependencies. They add checksums, a dependency graph, updates, vendoring, and arbitrary mounts. A module can be a full theme or one small capability. Remote modules require Go and Git; local themes do not (using modules).

Operationally useful commands include:

```
hugo mod graph
hugo mod verify
hugo mod vendor
hugo config mounts
```

Commit `go.sum`. Vendoring makes dependency content available for inspection and offline builds, but `_vendor` is generated dependency material: do not edit it as the customisation mechanism. Override from the project or maintain a proper fork.

Mounts map arbitrary paths into Hugo's component namespaces. For example, a design-system package could mount its CSS beneath `assets/`. Defining any mount for a component removes that component's default mount, so a project that adds one custom content mount must re-add the ordinary `content → content` mount if it still needs it. Current mount configuration uses `files` and `sites`; the older `includeFiles`, `excludeFiles`, and `lang` keys were deprecated in 0.153.0 and have since been removed, so in 0.168.0 a config that still uses them fails the build (`module.mounts.lang was deprecated in Hugo v0.153.0 and subsequently removed`) rather than merely warning (module configuration).

Mounts are excellent for presenting a clean virtual structure. They can

also make ownership invisible. Review `hugo config mounts` and trace a winning file back to its physical source before editing.

7.6 Configuration merging is deliberately constrained

Project configuration has precedence over themes and modules. `_merge` can select `none`, `shallow`, or `deep` behaviour for supported maps. Slices cannot be merged generically, and most categories default to no merge as a security measure. A permissive root-level `_merge` can let a dependency influence sensitive categories such as markup or security (configuration merging).

Theme parameter names should be namespaced. A generic `params.enabled` is likely to collide; `params.organisationHeader.enabled` communicates ownership. Treat a theme's configuration schema like a public API: document defaults, validate required values where possible, and make unsupported combinations fail visibly.

7.7 How to inspect an inherited theme

Before directing a change, establish:

1. The pinned theme/module version and supported Hugo range.
2. The effective component mounts and dependency graph.
3. The winning template for representative routes.
4. The theme's content and parameter schema.
5. Existing partials, blocks, CSS tokens, and render hooks intended as seams.
6. External build tools and network calls.
7. Current warnings, deprecations, security posture, and upgrade history.
8. Output behaviour under a production build, not only `hugo server`.

Then classify the requested change as configuration, an additive project file, an exact override, a reusable component, or a fork. This turns "customise the theme" into a bounded engineering decision.

Exercise 6 — choose the theme boundary

Green Horizon wants a shared accessibility-tested navigation component across six sites. Each site has distinct campaign templates and branding. The current theme also supplies a navigation partial, but its DOM is unsuitable. Compare an exact override in every project, a narrow shared component placed before each base theme, and a fork of every theme. What compatibility contract would the shared component need?

Worked answer

A useful question for an implementation agent

> Identify the smallest stable seam for this change. Show the dependency and lookup evidence, list copied or shadowed files, and explain how an upstream update will be reviewed.

A strong answer minimises shadowed surface while naming the contract it accepts. A weak answer copies the entire theme into the project because that makes the immediate edit easy.

8. Publish more than web pages

8.1 One page can have several outputs

Lens Vault defines a custom JSON format and enables it for home:

```
[outputFormats]
  [outputFormats.Catalog]
    mediaType = 'application/json'
    baseName = 'catalog'
    isPlainText = true
    notAlternative = true

[outputs]
  home = ['HTML', 'Catalog']
```

The `layouts/home.catalog.json` template produces `/catalog.json` from the same generated album pages used by HTML:

```
{{- $albums := slice -}}
{{- with site.GetPage "/albums" -}}
  {{- range .Pages -}}
    {{- $albums = $albums | append (dict
      "title" .Title
      "url" .RelPermalink
      "summary" .Summary
      "tags" .Params.album.tags
    ) -}}
  {{- end -}}
{{- end -}}
{{- dict "generated" now "albums" $albums | jsonify -}}
```

Output formats can be configured by page kind or added in front matter. They have media types, suffixes, base names, paths, and link behaviour (output formats, outputs). Useful projections include search indexes, partner feeds, web manifests, calendars, and constrained data exports.

A second output is not automatically free. It is another public interface with schema, size, caching, privacy, and compatibility requirements. Do not serialize all front matter into JSON because it is available. Define a public schema and test absence as well as presence of fields.

The `generated` timestamp makes Lens Vault's catalogue intentionally non-reproducible byte-for-byte, even under `--clock`, which sets only the starting time. It communicates freshness, but it also breaks content-addressed comparisons on every build. An alternative is the immutable snapshot's source timestamp or digest. Choose which truth consumers need.

8.2 Render hooks turn Markdown constructs into site policy

A link render hook can reject broken internal references, mark external links, or add language-aware behaviour. An image hook can find a page resource, generate renditions, add dimensions, enforce alternative text, and wrap a caption. Code-block and heading hooks can enforce documentation conventions.

Hooks belong under `layouts/_markup` and participate in page-aware

lookup. They are often a better long-term content interface than shortcodes because authors keep writing ordinary Markdown. The boundary is semantic: use a hook when the standard Markdown construct has a site-wide rendering policy; use a shortcode when authors need to invoke a genuine domain component with parameters.

For example, Green Horizon could make ordinary Markdown links to evidence PDFs emit file type, size, and review date derived from the resource. That improves accessibility and traceability without requiring every editor to remember presentation markup.

8.3 Multilingual output is another site dimension

Hugo can relate translations by filename suffix, language-specific content directories, or `translationKey`, and can generate single-host or multihost sites. Templates should use page permalinks and language-aware URL helpers rather than hand-building paths. Bundle resources also participate in language selection and fallback (multilingual sites).

Translation is not string substitution over finished HTML. Decide:

- whether content identity is shared across languages;
- whether taxonomy terms and URL slugs are localised;
- which resources have language variants;
- how missing translations fall back or remain absent;
- whether a component's layout expands safely for different scripts; and
- whether alternate outputs carry the same language semantics.

A theme advertised as multilingual should demonstrate navigation, taxonomies, dates, structured data, and alternate links—not merely a translated button.

8.4 Headless bundles separate storage from routing

Build options independently control whether a page is listed, rendered, and whether its resources publish. A headless bundle remains retrievable to templates while producing no ordinary page. This is useful for reusable policy fragments, shared campaign calls to action, or a content-managed home-page composition. Explicit build options are more general than the older `headless` shortcut (build options).

Do not make content headless merely because the current design does not link to it. Headless content loses its own route and can create accessibility, preview, and editorial-understanding problems. Use it when the content's identity is truly as an embedded resource or projection.

Exercise 7 — design a partner output

Green Horizon's coalition partners need a JSON feed of active public campaigns, including stable ID, URL, title, issues, places, and last evidence-review date. Draft the acceptance criteria. Which page data must not leak? What should happen when one evidence file is malformed? How will consumers learn about a schema change?

Worked answer

A useful question for an implementation agent

> Treat this output as an API: show its schema, source mapping, representative artefact, deterministic ordering, privacy exclusions, and compatibility test.

A strong answer validates the generated JSON independently and versions or otherwise governs the schema. A weak answer says `jsonify .` and exposes the entire internal page object or parameter map.

9. Operate and diagnose the build

9.1 Development server, production build, and deployment are distinct

`hugo server` watches inputs, rebuilds, serves from a development environment, and injects LiveReload. It is trusted local tooling, not a production server. Production correctness needs a production build with the real base URL, environment, dependency set, minification decisions, and destination policy (security model).

Likewise, `hugo build` publishes files but does not prove deployment. An atomic hosting workflow is conceptually:

```
resolve pinned inputs
  → build into a fresh directory
  → inspect and test that directory
  → upload/stage all artefacts
  → promote the complete version
  → retain a previous complete version for rollback
```

If a build fails, discard its destination. If upload fails before promotion, the current site remains. If post-promotion monitoring fails, promote the last known-good artefact. Reverting source and rebuilding is not an exact rollback when remote data, time, dependencies, or tool versions have changed.

Hugo's optional `hugo deploy` only targets Amazon S3, Azure Blob Storage, and Google Cloud Storage, and requires a deploy edition. It does not replace a general deployment design (deploy with Hugo).

9.2 A diagnostic sequence starts from the symptom

Use evidence to distinguish causes before editing:

Symptom	Competing causes	Discriminating evidence
Wrong layout	Old layout path; type override; more-specific theme template; wrong output	Page kind/type/path/output; `templates.Current`; component mounts
Page absent	Draft/future/expired; adapter filter; branch/leaf mistake; build option; path collision	`hugo list`; adapter input; `--printPathWarnings`; page graph dump
Old page remains	Destination was not cleaned; deployment retained old object	Fresh-directory build versus deployed object inventory
Remote data seems stale	File cache TTL; HTTP policy; watch polling disabled; upstream cache	Cache config, response metadata, restart/clean-cache comparison

Symptom	Competing causes	Discriminating evidence
Asset missing	File is under `assets` but never published; wrong mount; pipeline error	Resource lookup result, effective mounts, generated HTML and output tree
Partial varies incorrectly	Incomplete `partialCached` variant key; hidden global/page state	Uncached comparison, call contexts, cache keys
Development works, CI fails	Hugo/tool version; case-sensitive path; missing module/tool; network policy	`hugo env`, lock/checksum files, clean CI log, filesystem paths
Build succeeds but site is wrong	Wrong base URL; missing routes; private fields leaked; invalid HTML; inaccessible interaction	Independent output and browser-level acceptance tests

Useful commands and flags include:

```
hugo env
hugo config
hugo config mounts
hugo build --logLevel debug
hugo build --printPathWarnings --printUnusedTemplates
hugo build --panicOnWarning
hugo build --templateMetrics --templateMetricsHints
hugo build --printMemoryUsage
hugo build --clock 2026-10-21T12:00:00+02:00
```

`debug.Dump`, `warnf`, and `templates.Current` help inspect templates. Remove temporary diagnostic output after establishing the cause. `--clock` makes date-sensitive inclusion and `now` predictable in a test (inspection, build command).

Diagnostics need interpretation. `--printUnusedTemplates` lists tem-

plates that no page selected; it printed nothing for the photo fixture, whose `home.catalog.json` is used. When a report contradicts the generated artefacts, for example by naming a template whose output exists, that is reason to investigate or report the diagnostic, not to delete a working template. Conversely, a noisy false positive is not reason to ignore every warning. Reconcile each warning with a minimal case and observed output.

9.3 Performance evidence has Hugo-specific traps

Template metrics can identify frequently called or expensive templates and can suggest partial caching. A 100% cacheability hint only says observed rendered output matched across calls. It cannot see logging, validation side effects, or hidden dependence on order or state. Rendering is parallel, so cumulative template time can exceed wall-clock time (performance troubleshooting).

Optimise after measuring a representative production build. A complicated cached partial with incomplete keys is worse than a fast correct build. For large photo sites, measure page count, API snapshot size, image transformation count, peak memory, warm and cold build time, and output size separately.

9.4 Know which inputs are trusted

Hugo documents layouts, assets, archetypes, cached resources, data, translations, and static files as trusted. Content and content-adapter output are untrusted, except that enabling inline shortcodes changes the content trust boundary. Hugo restricts filesystem access, external commands, environment variables, and remote HTTP through security policies (security model, security configuration).

This has practical consequences:

- A third-party theme is trusted build code. Review and pin it.
- `safe.HTML`, unsafe Markdown rendering, or inline shortcodes are trust decisions, not formatting conveniences.
- A broad `_merge` policy may let dependencies alter security-sensitive config.
- Remote HTTP allowlists should be narrow. Do not weaken local/private-address protection to make a development endpoint

convenient in production.
- Build credentials should have least privilege, should not enter published data, errors, or cache artefacts, and should not be available to unreviewed theme code.
- A static result can still contain XSS, unsafe third-party JavaScript, leaked metadata, or broken access-control assumptions.

The v0.168.0 release includes relevant Go HTML-template, HTTP, and minifier security fixes, so the tutorial pins it rather than treating all 0.x versions as interchangeable (v0.168.0 release).

9.5 Upgrade with deprecations promoted to evidence

Run an upgrade in isolation, read the intervening release notes, perform a clean production build, and resolve deprecations rather than suppressing them. Hugo's current deprecation process logs notices before removal (deprecations).

Version-sensitive traps relevant to inherited sites include:

- pre-0.146 layout paths;
- `.Site.Data` instead of `hugo.Data`;
- `Page.Scratch` where the documentation now presents `Page .Store` (0.168.0 emitted no deprecation notice for it in a test build, so treat this as a modernisation, not a failure);
- obsolete remote-resource `.Err` handling instead of `try`;
- Extended/LibSass assumptions;
- removed mount keys (`includeFiles`, `excludeFiles`, `lang`), which now fail the build;
- image `.Exif`, deprecated in 0.155.0 in favour of `.Meta`; and
- `resources.PostProcess`, deprecated in 0.164.0 in favour of `templates.Defer`.

Do not mechanically modernise unrelated code during a theme change. Establish the current warning baseline, keep the upgrade diff reviewable, and test the routes and outputs whose lookup may have changed.

Exercise 8 — diagnose rather than thrash

After a theme upgrade, campaign pages use the generic visual treatment but home still looks correct. The content files and URLs have not changed. List at least four plausible causes, the evidence that distinguishes them, and the smallest safe recovery if a production deadline is near.

Worked answer

A useful question for an implementation agent

> Reproduce the symptom from a clean production build, name the first divergent artefact or dispatch decision, and show why the proposed change addresses that cause without broadening the diff.

A strong answer preserves the failed artefact or log, compares effective versions/configuration, and verifies representative output after the change. A weak answer repeatedly clears caches or copies templates until the appearance returns.

10. Review and verify a change

10.1 A successful build proves only a subset

Exit status zero establishes that Hugo parsed enough configuration, content, templates, and resources to complete. It does not establish that:

- the intended template won;
- every required route exists and no forbidden route exists;
- links and canonical URLs target the deployment location;
- generated HTML is valid, accessible, or responsive;
- JSON obeys its public schema;
- private input fields or source resources were not published;
- ordering, translations, image crops, and empty states are acceptable;
- a second build removes deleted pages; or
- deployment is complete and reversible.

Verification should follow the same layers as the system.

10.2 A proportionate evidence stack

1. **Input and dependency checks** establish the Hugo version, theme/module versions, checksums, toolchain, schema, and configuration.
2. **Strict clean build** catches parse/execution failures, deprecations promoted to warnings, path collisions, and some unused templates.
3. **Structural output checks** assert routes, absence of drafts/private data, asset hashes, JSON schemas, sitemap/feed membership, and stale-file removal.
4. **Semantic checks** assert correct campaign grouping, album ordering, evidence review dates, language relationships, and fallback behaviour.
5. **Standards and accessibility checks** inspect HTML, links, headings, names, alternative text, contrast, and keyboard interactions with appropriate tools.
6. **Representative browser checks** exercise layout, responsive images, JavaScript, and real backend interactions where present.
7. **Deployment checks** validate the actual host's headers, redirects, caching, base paths, atomic promotion, and rollback artefact.

Hugo documents a production site-audit procedure that builds with diagnostic placeholders enabled and scans the output for known failure markers. It is a shell procedure, not a `hugo audit` subcommand, and it does not constitute a complete site test suite (site audit). The exact external tools should fit the site's risk and stack.

10.3 Test invariants, not implementation trivia

Good acceptance checks for Green Horizon include:

- every published campaign has a non-empty evidence register;
- issue and place term pages contain exactly their member campaigns;
- a high-priority campaign renders the project override's urgency marker;

- source evidence JSON is either intentionally published or explicitly suppressed;
- no draft or expired action appears in feeds or sitemaps; and
- internal campaign references resolve in every language.

Good Lens Vault checks include:

- one public adapter record produces one stable album route;
- duplicate or invalid album IDs fail before ambiguous output;
- `/catalog.json` validates against its public schema and has deterministic ordering;
- only public rendition URLs, never originals or signed URLs, are emitted;
- width/height, `srcset`, and alternative text match the snapshot;
- API outage behaviour follows the stated snapshot policy; and
- authenticated edits are tested against the runtime service, not mistaken for Hugo behaviour.

Snapshotting whole pages can help detect unplanned change, but focused invariants explain impact and survive harmless formatting changes. Combine structural assertions with a small number of visual regressions where design matters.

10.4 Review plausible generated mistakes

Watch specifically for code that looks idiomatic but rests on the wrong model:

Mistake	Why it misleads	Detection
Uses `_default/single.html` as the taught primary layout	Mixes the legacy and current systems	Compare against v0.146+ docs; inspect winning template
Uses top-level `type` as arbitrary metadata	Changes template family	Dump `.Type`; inspect lookup candidates
Puts API credentials or private records in published JSON/front matter	Static output has no access control	Search complete output and caches for forbidden markers

Mistake	Why it misleads	Detection
Uses `partialCached` without all variants	First cached context leaks into other pages	Compare uncached outputs across pages/languages
Adds a mount but omits the default component mount	Existing content/assets disappear	`hugo config mounts`; route inventory
Forks the complete theme for one card	Hides upstream maintenance cost	Count copied files and compare available seams
Uses `--gc` to remove a deleted public page	Cleans cache, not destination	Fresh-directory versus repeat-build output
Treats `hugo server` as deployment proof	Uses development serving and environment	Run clean production build and host-level checks
Relies on global mutable template state order	Rendering is concurrent	Repeat/stress build and redesign as graph query
Calls the photo API at browser runtime with build credentials	Moves secrets into the client	Inspect generated JS/network requests

10.5 Decide what to delegate and what to retain

Routine template syntax, CSS implementation, mechanical content migration, fixture generation, and check scripting are good candidates for delegation once their contracts are explicit.

Retain or explicitly approve:

- the page graph, stable identities, and URL migration policy;
- static snapshot versus runtime boundaries;
- theme/module ownership and upgrade strategy;
- public output schemas and content trust;
- remote failure/freshness policy and build credentials;
- accessibility and browser support targets;
- production acceptance evidence; and
- deployment promotion and rollback semantics.

This is not because templates are mysterious. It is because these decisions determine which system owns truth and what future changes cost.

Exercise 9 — review the whole proposal

An agent proposes a photo site that imports a popular gallery theme as an unpinned Git submodule, puts a long-lived API token in `params`, fetches albums directly during every Hugo render, caches the gallery partial without variant keys, and deploys `public/` in place after `hugo server` looks correct. Rank the problems by impact and specify the minimum evidence required before release.

Worked answer

A useful question for an implementation agent

> Give me the acceptance matrix: requirement, source of truth, check, observed evidence, and what remains unproved. Include failure and second-run cases.

A strong answer separates build, semantic, browser, and deployment evidence and is candid about limits. A weak answer offers only the build transcript and a screenshot.

11. Worked reasoning

Try the exercises before reading this section. These are reasoned solutions, not unique designs.

Exercise 1

Back to exercise 1

Hugo can fetch or consume the set of publishable albums, create routes and indexes, and publish a snapshot. It cannot receive an authenticated edit or make that edit immediately visible after the build exits. The backend owns identity, authorisation, mutation, and read-after-write behaviour; the browser may call it. A workable freshness contract

is, for example, “a successful publish webhook makes approved public changes visible within ten minutes; the site identifies its snapshot; failed rebuilds leave the previous complete site live.” That contract implies a trigger, maximum build/queue time, monitoring, and atomic promotion.

Exercise 2

Back to exercise 2

An `events/` section is a strong default because events have independent identity, routes, cancellation state, and queries across campaigns. Each event can be a leaf bundle if it owns documents or images. Venue and accessibility details are parameters or a referenced venue page, depending on whether venues are reused and independently maintained. Campaign should be an explicit stable reference because “the campaign for this event” has cardinality and integrity; issue/place taxonomies may be useful for cross-cutting discovery. Nesting events inside campaigns makes campaign-local ownership convenient but complicates an all-events calendar and moving an event between campaigns. Choose it only if campaign ownership dominates and stable URLs survive moves.

Exercise 3

Back to exercise 3

The project partial wins because it has the same relative component path at higher precedence, even though the call originates in a theme template. Adding `layouts/campaigns/page.html` changes regular pages whose logical path/type selects the campaign family; home and campaign section do not select it. The theme’s `baseof.html` can still wrap it if the new page template contains one or more definitions and nothing else except whitespace and template comments.

Exercise 4

Back to exercise 4

Keep a validated last-known-good snapshot as an immutable build input with fetch time, source revision/ETag, schema version, and digest.

A scheduled or webhook fetch attempts an update. A transient failure reuses the snapshot only while its age remains below an explicit threshold and raises an operational warning; once 24-hour deletion compliance can no longer be guaranteed, fail publication and alert rather than silently extending staleness. Malformed data never replaces the known-good snapshot. The deployment manifest records the snapshot digest. Test initial absence, timeout, 404, invalid schema, reuse within the window, and expiry beyond it.

Exercise 5

Back to exercise 5

Reject the proposed transformation. It asks the build to access private originals, duplicates an authoritative rendition pipeline, multiplies network and CPU work, and couples every site deployment to every original's availability. Use the immutable public renditions and dimensions from the backend contract. If Hugo needs editorial crops unavailable from the backend, make that a deliberate new backend rendition or a bounded local-resource case; do not broaden access to all originals by default.

Exercise 6

Back to exercise 6

Six copied overrides create six upgrade points. Six theme forks are broader still. A narrow navigation component placed before each base theme is attractive if it can define a stable contract: required menu data, active-page input, language behaviour, DOM/accessibility guarantees, CSS token interface, and a documented way for bases to call or yield to it. Confirm that existing themes do not hard-wire incompatible header structure. Where they do, a one-file bridge override per theme may call the shared component; this keeps base-theme forks unnecessary.

Exercise 7

Back to exercise 7

Define a versioned public record schema and deterministic sort, preferably by a stable ID. Assert that only active, publishable campaigns appear; referenced terms resolve; every record has a canonical URL; and the latest evidence-review date derives from valid evidence. Malformed required evidence should fail the feed/site build rather than publish a falsely authoritative record. Exclude internal owners, drafts, embargo dates, notes, source paths, and any credentials. Publish schema/version documentation and use consumer-driven compatibility tests or an explicit versioned endpoint for breaking changes.

Exercise 8

Back to exercise 8

Plausible causes include: the upgrade moved from legacy to current layout paths; campaign `type` now selects a different family; a more-specific theme template was added and outranks the project fallback; the project override's relative path no longer matches; or a configuration merge changed. Inspect the affected page's kind/type/path/output, `templates.Current`, effective mounts, and dependency diff. Near a deadline, pin the last known-good theme version and rebuild from the last known-good inputs. A narrow exact override is acceptable only after identifying the new winning path; copying the whole old theme is not the smallest safe recovery.

Exercise 9

Back to exercise 9

The long-lived token in published or theme-visible configuration is highest impact because trusted theme code may read it and generated artefacts may leak it. In-place deployment without a validated complete artefact risks partial or stale production output. Unpinned dependencies undermine reproducibility and supply-chain review. Unbounded per-render API fetching creates availability, rate, consistency, and build-time risks. Incorrect partial caching corrupts pages context-dependently. Replace these with least-privilege build credentials outside publishable config, a pinned/reviewed dependency, one validated snapshot per build, correct cache variants or no partial cache, a clean strict production build, structural/privacy/browser tests, and atomic

promotion with rollback.

12. Fixture guide

The examples live beside this tutorial:

```
examples/
├── advocacy/       # local theme + project partial override +
 ↪  page resources
└── photo-front/    # project templates + content adapter +
 ↪  custom JSON output
```

Install Hugo 0.168.0 from the official release, then from this `hugo/` tutorial directory run:

```
hugo build \
  --source examples/advocacy \
  --destination /tmp/hugo-advocacy-public \
  --noBuildLock \
  --printPathWarnings \
  --panicOnWarning

hugo build \
  --source examples/photo-front \
  --destination /tmp/hugo-photo-public \
  --noBuildLock \
  --printPathWarnings \
  --panicOnWarning \
  --clock 2026-10-21T12:00:00+02:00
```

Use explicit temporary destinations so tutorial builds do not create untracked `public/` directories. Expected observations include:

- advocacy output has campaign, issue, and place routes;
- campaign pages contain evidence read from bundle-local JSON;
- high-urgency cards have the project override's `data-urgency="high"`;
- the stylesheet filename is content-fingerprinted;
- photo output has two pages generated by the content adapter;
- `/catalog.json` contains the same two albums and a `generated` value that starts at the `--clock` time but carries a few millisec-

onds of drift, so the file differs between builds; and
- photo markup uses backend-provided responsive rendition URLs.

Useful inspection commands:

```
hugo config --source examples/photo-front
hugo config mounts --source examples/advocacy
find /tmp/hugo-photo-public -type f -print
```

The example sites contain no live backend and their rendition hosts are the reserved `.example` domain. They test generated markup, not image delivery or authenticated behaviour. See the verification record for what was executed and what remains unproved.

13. Compact field reference

Choose the mechanism

Need	Start with
One authored page with related assets/data	Leaf bundle (`index.md`) and page resources
Navigable branch with descendants	Branch bundle (`_index.md`)
Many-to-many browsing by controlled vocabulary	Taxonomy
Pages derived from a structured catalogue	Content adapter
Small shared structured data	`data/` and `hugo.Data`
Large/on-demand local structured input	Global resource under `assets/`
Ordinary content element with site policy	Render hook
Explicit domain component in prose	Shortcode
Named reusable template component	Partial
Page-aware polymorphic card/view	`Page.Render` view
One-file change to inherited theme	Exact project override

Need	Start with
Reusable narrow capability	Theme component or Hugo Module
Live authenticated mutation	Backend/browser application, not Hugo

Retained causal rules

- `index.md` is a leaf; `_index.md` is a branch.
- Kind, section, type, layout, taxonomy, and term are different axes.
- Current layouts are rooted directly in `layouts/`; partials use `_partials`.
- Template specificity and component precedence jointly select the winner.
- A base applies only to definition-only templates.
- `.` changes inside `range` and `with`; partial context is explicit.
- `assets` are pipeline inputs; `static` files copy through.
- Page resources are bundle-scoped.
- A remote build input needs freshness, failure, schema, credential, and cache policies.
- A theme is trusted code and a content-schema dependency.
- Adding a component mount removes that component's default mount.
- `--gc` cleans caches; destination cleaning removes stale published files.
- `hugo server` is a development aid; inspect a clean production artefact.
- A zero exit status is necessary evidence, never the whole acceptance case.

Sources and version boundary

Consequential claims link to the official Hugo 0.168-era documentation beside the relevant text. The principal version boundary is the template overhaul in 0.146.0; material teaching `_default`, `partials`, or a home `index.html` as the primary current structure predates that

model. The target release is Hugo 0.168.0, released 13 October 2026. Future upgrades should be checked against intervening release notes and the official deprecation guidance rather than assumed compatible.